

version control

Version Control Concepts – Definition & Purpose

1. Definition of Version Control

Version Control is a system that **records changes made to files over time**, so you can:

- Look back at previous versions
- Compare differences
- Restore an earlier state
- Track who made which change

Think of it like a **Time Machine for your project code**.

2. Purpose of Version Control (Why we need it)

Version Control solves real problems in software development.

Here's what it helps with:

♦ 1. Avoid Losing Work

Every change you commit is saved, so even if your machine crashes or code breaks, you can always return to a safe version.

♦ 2. Work in Teams Without Conflicts

Multiple developers can work on the same project **at the same time**.

No overwriting other people's code.

Example:

- Developer A works on login page
- Developer B works on payment page Both can commit without disturbing each other.

♦ 3. Track Who Did What

Version Control keeps a history of:

- Who changed the file
- What exactly changed
- When it changed
- Why it changed (commit message)

This makes debugging easier.

Benefits of Version Control

1 Prevents Data Loss & Keeps Complete History

Version control stores every version of your project.
If something breaks, gets deleted, or a feature fails,
you can instantly restore an older working version.

Simple Example:

Your project works today but breaks tomorrow—no problem.
Just go back to yesterday's version.

2 Enables Team Collaboration Without Conflicts

Multiple developers can work on the same project at the same time
without overwriting each other's work.

Simple Example:

Developer A works on the homepage
Developer B works on the cart page
Both work independently → merge later → no clashes.

Types of Version Control Systems

Version control systems are mainly classified into **three types**:

1 Local Version Control (LVS)

👉 **Definition:**

All version history is stored **on your local machine**.

👉 How it works:

You manually copy files into different folders to track versions.

👉 Real Scenario:

A developer saves project files like:

- `project_v1`
- `project_v2`
- `project_final`
- `project_final_final`

This is error-prone and easy to mess up.

Centralized Version Control System (CVCS)

👉 Definition:

Code is stored in **one central server**, and developers connect to that server.

👉 Examples:

- SVN
- Perforce
- CVS

👉 How it works:

Developers check out files from the central server → make changes → commit back to the same server.

👉 Real Scenario:

If the central server goes down,
nobody can work or access the project.

3 Distributed Version Control System (DVCS)

👉 Definition:

Every developer has a **full copy** of the entire repository (including history) on their own machine.

👉 Examples:

- Git
- Mercurial

👉 How it works:

Developers work independently on their local repository, then push/pull changes to a shared remote like GitHub.

👉 Real Scenario:

Even if GitHub goes down,
you still have the complete project on your laptop
→ so you can continue working.

Introduction to Git

Git is a fast and powerful **version control system** used to track changes in code and manage projects efficiently.

It helps developers save every version of their work, work together without conflicts, and safely experiment with new features.

Git as a Distributed Version Control System

Git is a **Distributed Version Control System (DVCS)**, which means every developer has a **complete copy** of the entire project, including its full history, on their own computer.

Why this is powerful?

- You can work **offline** without needing internet.
 - Even if the main server (like GitHub) goes down, your full project is still safe on your system
 - .Developers can create branches, commit changes, and experiment independently without affecting others.
 -]Collaboration becomes easy because changes can be shared through **push** and **pull** operations.
-

Git Components: Working Directory, Staging Area, Repository

1 Working Directory

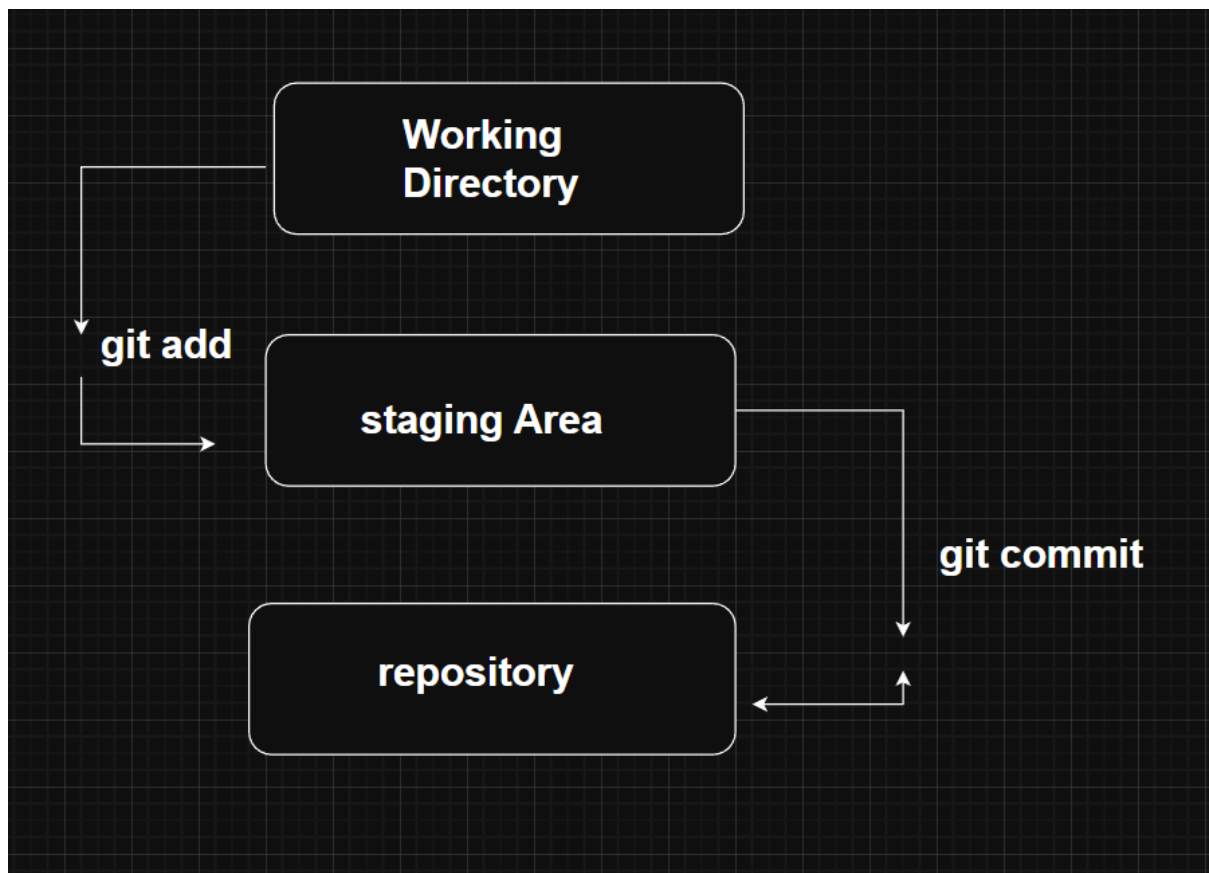
Where you create and edit files.
(Your current project workspace.)

2 Staging Area

Where you put files that are ready to be committed.
(Prepare changes using `git add`.)

3 Repository

Where Git permanently stores committed versions.
(Save history using `git commit`.)



Git Download & Installation (Simple Steps)

1 Download Git

- Go to the official website:
git-scm.com
 - Click **Download for Windows** (or Mac/Linux as needed).
-

2 Run the Installer

- Double-click the downloaded **.exe** file.
 - Click **Next** for default options (recommended for beginners).
-

3 Important Default Selections

- **Select editor** → Keep **Vim** or choose **Notepad** (optional)
- **Adjust PATH** → Use **Recommended** option
- **HTTPS option** → Keep default
- **Line ending** → Keep default
- **Terminal** → Git Bash (default)

Just keep clicking **Next** → **Next** → **Install**.

4 Finish Installation

- Click **Finish**
 - Open **Git Bash** from the Start menu.
-

5 Verify Installation

Open Git Bash and run:

```
git --version
```

Configuring Basic Git Settings

These settings tell Git **who you are**, so every commit is tagged with your identity.

1 Set Username

```
git config --global user.name "Your Name"
```

2 Set Email

```
git config --global user.email "yourmail@example.com"
```

3 Verify Settings

```
git config --global --list
```

You will see your username and email listed.

Creating a GitHub Repository

1 Log in to GitHub

Go to **github.com** and sign in to your account.

2 Create New Repository

- Click the **+** icon (top-right)
- Select **New repository**

3 Fill Repository Details

- **Repository Name** → (example: my-project)
- **Description** → optional
- **Visibility** → Choose **Public** or **Private**
Check **"Add README.md"** (optional but recommended)

4 Create Repo

Click **Create repository** → your repo is ready!

5 Copy the Repo URL

You will see HTTPS URL like:

<https://github.com/username/my-project.git>

This is used for cloning.

★ Initializing a New Git Repository

1 Open your project folder

Go to the folder where your project files are stored.

2 Initialize Git

Open **Git Bash** in that folder and run:

```
git init
```

This creates a hidden `.git` folder → Git starts tracking your project.

3 Verify Initialization

```
git status
```

 .git	06-12-2025 19:04	File folder	
 backend	06-12-2025 19:04	File folder	
 index	06-12-2025 19:04	Microsoft Edge HT...	1 KB
 style	06-12-2025 18:17	CSS Source File	1 KB

Cloning an Existing Repository

1 Copy Repository URL

From GitHub → click **Code** → copy the **HTTPS URL**
(example: `https://github.com/user/project.git`)

2 Open Git Bash

Go to the folder where you want to download the project.

3 Run Clone Command

```
git clone <repo-url>
```

Example: `git clone https://github.com/user/project.git`

Git Add – Staging Changes

1 What is Staging?

Staging means selecting which files are ready to be committed.

2 Stage a Single File

```
git add filename
```

3 Stage All Files

```
git add .
```

4 Purpose

`git add` moves changes from the **working directory** → **staging area**, preparing them for commit.

Here is a **short, sweet, and easy** note on **Using Wildcards in Git** 📌

★ Using Wildcards in Git

Wildcards help you **stage multiple files** matching a pattern.

1 Add all files with same extension

```
git add *.txt
```

Stages all **.txt** files.

2 Add files starting with a specific name

```
git add app*
```

Stages files like:

app.js, **app.css**, **app1.py**

3 Add files in a folder

```
git add src/*.js
```

Stages all **.js** files inside **src** folder.

In simple words:

Wildcards = pattern matching to quickly stage multiple files.

Git Commit – Committing Changes

1 What is Commit?

A commit is a **permanent save** of the changes that are in the staging area.

2 Commit with a Message

```
git commit -m "your message"
```

Git Status

1 Purpose

Shows the **current state** of your working directory and staging area.

2 What it displays

- Modified files
 - Untracked files
 - Staged files
 - Which branch you are on
 - What needs to be committed or added
-

3 Command

```
git status
```

Git Log – Viewing Commit History

1 Basic Command

```
git log
```

Shows the list of commits with:

- Commit ID
 - Author
 - Date
 - Message
-

★ Customizing Log Output (Simple Options)

2 One-line Log

```
git log --oneline
```

Shows each commit in a short, single line.

3 Graph View

```
git log --graph
```

Displays branch/merge structure visually.

4 One-line + Graph

```
git log --oneline --graph
```

Most commonly used for clean history.

5 Show last N commits

```
git log -n 5
```

Shows last 5 commits.

6 Show commits by a specific author

```
git log --author="John"
```

★ Introduction to Branching

1 What is a Branch?

A branch is a separate line of development where you can work on new features without affecting the main code.

2 Why Branching?

It allows you to:

- Develop features independently
- Fix bugs safely
- Experiment without breaking the main project
- Work in teams without conflicts

3 Simple Example

- main branch → stable, working project
 - feature-login branch → work on login feature
 - Once done → merge back into main
-

Creating and Managing Branches

1 Create a New Branch

```
git branch branch-name
```

Creates a new branch.

2 Switch to a Branch

```
git checkout branch-name
```

(or)

```
git switch branch-name
```

3 Create & Switch in One Step

```
git checkout -b branch-name
```

4 List All Branches

```
git branch
```

5 Delete a Branch

```
git branch -d branch-name
```

Deletes a branch (if merged).

6 Force Delete a Branch

```
git branch -D branch-name
```

Deletes even if not merged.

Merge Changes

1 Merging Branches

Used to bring changes from one branch into another (usually into `main`).

Step 1: Switch to the branch you want to merge into

```
git checkout main
```

Step 2: Merge the other branch

```
git merge branch-name
```

If changes don't conflict, Git merges automatically.

Handling Merge Conflicts

A merge conflict happens when **two branches change the same part of a file** and Git cannot decide which version to keep.

When conflict occurs

Git will show:

CONFLICT: file.txt

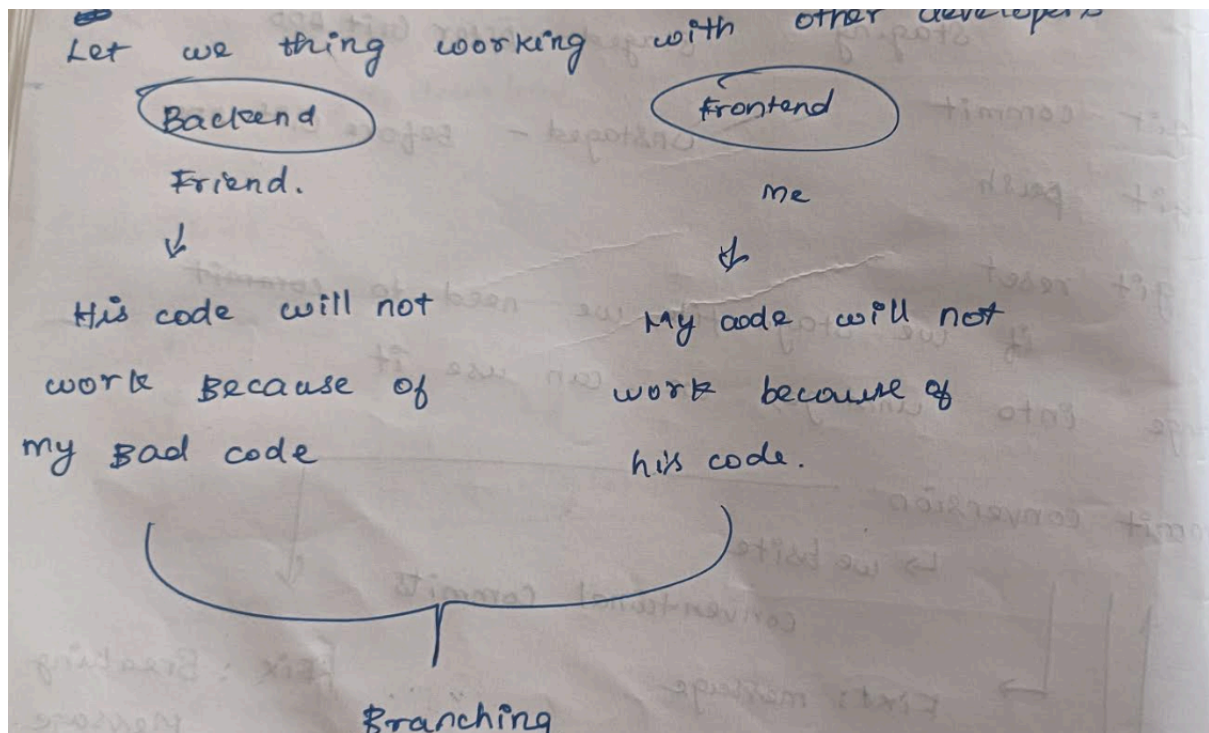
How to Solve

1. Open the file
2. Choose which code to keep (yours, theirs, or both)
3. Save the file
4. Add it again

git add file.txt

5. Complete the merge

git commit



```
<h1>I want this line</h1>
Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
<<<<<< HEAD (Current Change)
  <h1>I want this line</h1>
=====
  <h1>This is an important line</h1>
>>>>>> 0d83fb150af2d772316db474b7c8c30f4bd1e167 (Incoming Change)
</body>
</html>
```

Branching Strategies

1 Feature Branching

Used for working on **new features** separately.

How it works

- Create a branch for each feature
 `feature-login`, `feature-payment`
- Develop independently
- Merge into **main** once completed

Why use it

- ✓ Safe development
 - ✓ Easy code review
 - ✓ No disturbance to main code
-

2 Release Branching

Used when preparing a **new product release**.

How it works

- Create a branch like `release-v1.0`
- Final testing, bug fixes, polishing happens here
 After release → merge into **main** and **develop**

Why use it

- ✓ Clean release management
 - ✓ Keeps new features separate from final release work
-

3 Git Flow Workflow

A popular structured branching model.

Main Branches

- **main** → production-ready code
- **develop** → integration branch for ongoing work

Supporting Branches

- **feature branches** → new features
- **release branches** → preparing releases
- **hotfix branches** → quick fixes for production bugs

Why use it

- ✓ Best for medium/large teams
 - ✓ Clear structure for development, testing, and releases
-

Adding Remote Repository

1 Linking Local and Remote Repositories

You connect your local project to a remote server like GitHub using:

Add Remote

```
git remote add origin <repo-url>
```

Example:

```
git remote add origin https://github.com/user/project.git
```

Verify Remote

```
git remote -v
```

★ 2 Pushing Code to Remote

Once linked, upload your code:

```
git push -u origin main
```

The `-u` sets **origin/main** as the default for future pushes.

★ 3 Multiple Remotes

You can connect your local repo to more than one remote.

Add Another Remote

```
git remote add backup <url>
```

Example:

```
git remote add backup https://gitlab.com/user/project.git
```

Push to Backup Remote

```
git push backup main
```

Here is a **short, simple, and easy-to-understand** note on **Git Pull & Git Push** 📌

★ Git Push & Git Pull

1 Git Push – Upload Changes

Sends your **local commits** to the remote repository.

git push origin main

- `origin` → remote name
`main` → branch name
- Use `-u` first time to set upstream:

git push -u origin main

2 Git Pull – Download Changes

Fetches and merges **changes from remote** into your local repo.

git pull origin main

- Updates your local branch with the latest remote commits
 - Combines `git fetch` + `git merge` in one step
-

In Simple Words

- **Push** → send your work to remote
 - **Pull** → get the latest work from remote
-

Pushing Changes to Remote Repository

1 Purpose

Upload your **local commits** to a remote repository (GitHub, GitLab, etc.) so others can see or collaborate.

2 Command

```
git push origin branch-name
```

- `origin` → name of the remote
 - `branch-name` → branch you want to push (e.g., `main`)
-

③ First-Time Push with Upstream

```
git push -u origin main
```

- `-u` → sets **default upstream**, so future pushes can be done with just `git push`

Here is a **short, clear, and easy-to-understand** note on **Handling Remote Branches** 📌

★ Handling Remote Branches

① Tracking Remote Branches

When you clone a repo, Git automatically sets up **tracking branches**.
A tracking branch links your **local branch** to a **remote branch**.

View Remote Branches

```
git branch -r
```

Check Local Branch Tracking

```
git branch -vv
```

② Create a Local Branch from Remote

```
git checkout -b local-branch origin/remote-branch
```

- `origin/remote-branch` → remote branch

- `local-branch` → your new local branch
-

3 Push a Local Branch to Remote

`git push -u origin local-branch`

- Creates the branch on remote
 - Sets upstream for future pushes/pulls
-

Here is a **short, simple, and easy-to-understand** note on **Forking and Pull Request** 📌

★ Forking and Pull Request

1 Forking a Repository

- **Fork** = copy someone else's repository to your GitHub account.
- Lets you **experiment freely** without affecting the original repo.

Steps:

1. Go to the repo on GitHub
 2. Click **Fork** → now it's in your account
-

2 Making Changes

- Clone your fork locally

`git clone <forked-repo-url>`

- Create a branch, make changes, commit, push
-

3 Pull Request (PR)

- PR = request the **original repo owner** to merge your changes
- Lets the owner **review, discuss, and merge** your contribution

Steps:

1. Push changes to your forked branch
 2. Click **New Pull Request** on GitHub
 3. Select your branch → target branch → submit
-

Here is a **clean, simple, beginner-friendly note** on **Creating and Managing Pull Requests** 👉

★ Creating and Managing Pull Requests (PRs)

1 What is a Pull Request?

A **Pull Request (PR)** is a request to merge your code changes into another branch or another repository.

PR is used for:

- ✓ Code review
 - ✓ Collaboration
 - ✓ Suggesting improvements
 - ✓ Version control in teams
-

2 Creating a Pull Request

Step 1: Push your branch to GitHub

git add .

```
git commit -m "your message"
```

```
git push origin <branch-name>
```

Step 2: Go to GitHub

- Open your repository
- GitHub will show “**Compare & Pull Request**”
- Click it
(Or click **Pull Requests** → **New Pull Request**)

Step 3: Select branches

- **Base branch** → where you want to merge (example: `main`)
- **Compare branch** → your working branch

Step 4: Add PR details

- Title
 - Description (what changes you made, why)
 - Attach screenshots if UI changes
 - Click **Create Pull Request**
-

Managing a Pull Request

 **After creating the PR, you or reviewers can:**

✓ **Review Code**

- Add comments
- Suggest changes

- Request modifications

✓ Update PR

If changes are requested, update code locally → push again

git push origin <branch-name>

PR updates automatically.

✓ Resolve Merge Conflicts

If code conflicts:

- GitHub shows conflicts
- Fix file locally
- Commit & push
- Conflicts will clear

✓ Approve or Request Changes

Reviewers can:

- **Approve** → ready to merge
- **Request changes** → you must fix

4 Merging the Pull Request

Once approved:

Options:

- **Merge Commit**
- **Squash and Merge** (clean history)

- **Rebase and Merge**

Click **Merge Pull Request** → **Confirm Merge**

5 Closing a Pull Request

If not needed, you can **Close PR** without merging.

Quick Summary

Step	Action
1	Push branch
2	Open PR on GitHub
3	Select base + compare branches
4	Add title & description
5	Submit PR
6	Review, fix conflicts, update
7	Merge or close
